Report

BOT DESIGN

Kriti Inter-Hostel Tech Competition 2026

Hostel ID – 2669

March 11, 2026

Contents

1 Hardware **3**
1.1 Hardware Components . . . 3
1.2 Block Diagram . . . 3
1.3 System Architecture . . . 4
1.3.1 1. Sensing Layer . . . 4
1.3.2 2. Processing Layer . . . 4
1.3.3 3. Actuation Layer . . . 4
1.3.4 4. Visualization Layer . . . 4
1.3.5 Signal Flow Summary . . . 5
1.4 Data Flow / Assembly Description . . . 5
1.4.1 Sensor Assembly . . . 5
1.4.2 Motor Driver Assembly . . . 5
1.4.3 Camera and Servo Assembly . . . 5
1.4.4 Power Distribution . . . 5
1.5 Schematic . . . 5
1.6 Connection Description . . . 6

2 Software **7**
2.1 Software Used . . . 7
2.2 Line Following . . . 7
2.2.1 IR Sensor Array and Weighted Error . . . 8
2.2.2 PID Control for Smooth Line Tracking . . . 8
2.2.3 Line Recovery Function . . . 8
2.2.4 Non-Blocking Obstacle Avoidance . . . 8
2.2.5 Motor Driver Interface . . . 9
2.2.6 Overall Control Flow . . . 9
2.3 Image Detection and Identification Pipeline . . . 9
2.3.1 Model Training . . . 9
2.3.2 Image Acquisition . . . 10
2.3.3 Preprocessing . . . 10
2.3.4 Region of Interest (ROI) Selection . . . 10
2.3.5 Object Detection . . . 10
2.3.6 Feature Extraction . . . 11
2.3.7 Object Identification and Classification . . . 11
2.3.8 Post-Processing and Action Trigger . . . 11
2.4 Web Dashboard . . . 12
2.4.1 File structure . . . 12
2.5 Usage Documentation . . . 13
2.5.1 1. Uploading the Arduino Control Program . . . 13
2.5.2 2. Powering and Booting the Raspberry Pi . . . 14
2.5.3 3. Launching the Vision and Web Interface . . . 14
2.5.4 4. Accessing the Web Dashboard . . . 14
2.5.5 5. Starting Robot Operation . . . 15
2.5.6 6. Card Detection Process . . . 15
2.5.7 7. Resetting and Restarting . . . 15

2.6 Bill of Materials (BOM) 16

1 Hardware

1.1 Hardware Components

The following hardware components are used in the system:

1. **Raspberry Pi 4** A single-board computer used for high-level processing. It handles image processing, object detection, and camera control in the system.
2. **Arduino Microcontroller** The Arduino is responsible for real-time control of the robot. It reads data from sensors and generates control signals for the motor driver to perform line following.
3. **IR Sensor Array** The infrared sensors detect the contrast between the black line and the white surface. The sensor outputs are used by the Arduino to determine the robot's position relative to the path.
4. **Ultrasonic Sensor** This sensor is used for obstacle detection. It measures the distance between the robot and objects in front of it using ultrasonic waves.
5. **Motor Driver (L298N)** The motor driver acts as an interface between the Arduino and the DC motors. It receives low-power control signals from the Arduino and supplies sufficient current to drive the motors.
6. **DC Motors** The DC motors provide mechanical movement for the robot. Their speed and direction are controlled through the motor driver.
7. **Servo Motor** The servo motor is used to rotate the camera module so that it can track detected objects.
8. **Camera Module** The camera captures images of the surrounding environment. These images are processed by the Raspberry Pi for object detection and identification.
9. **Buck Converters** Two buck converters are used for voltage regulation. Ubec 5V/3A converter powers the Raspberry Pi, while 5V buck converter supplies regulated power to the Arduino and sensors.
10. **Battery** / **Power Supply** The 11.1V LiPo Battery supplies energy to all electronic components through the buck converters.

1.2 Block Diagram

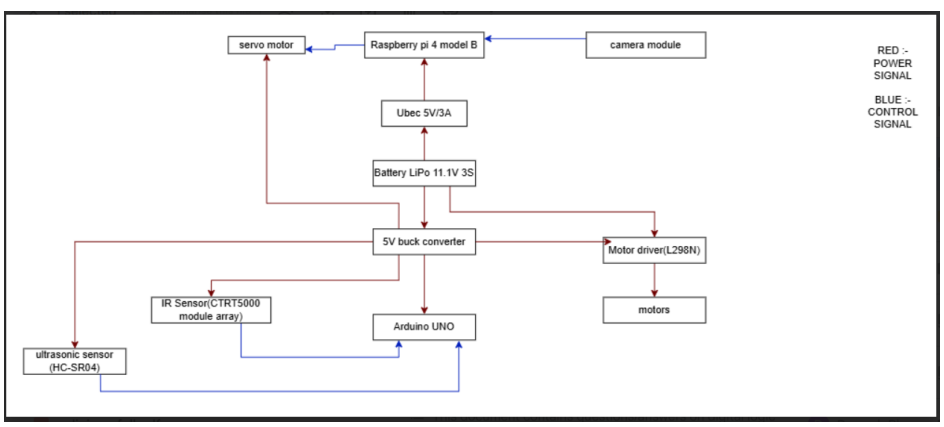

Figure 1: Block diagram of the system

1.3 System Architecture

The semi-autonomous robot is designed with modular sensing, processing, actuation, and visualization layers. Each layer interacts efficiently to ensure precise line following, object detection, and responsive servo control.

1.3.1 1. Sensing Layer

- **IR Sensor Array:** Detects the black line on the track. Provides analog signals to the Arduino for line-following control.
- **Ultrasonic Sensors:** Two ultrasonic sensors (left and right) are mounted on the robot. At any time, only one sensor detects the presence of an image card on its side. The detecting sensor sends a signal to the Arduino to indicate the location of the card.

1.3.2 2. Processing Layer

- **Arduino Microcontroller:**
  - Receives input from IR sensors and ultrasonic sensors.
  - Calculates line position error using weighted sensor readings.
  - Computes PID output to control motor speeds and servo rotation.
  - Commands the servo motor to rotate toward the side where a card is detected.
- **Raspberry Pi:**
  - Captures images from the camera module.
  - Runs lightweight object detection (YOLO) on captured frames.
  - Sends processed detection data to the web dashboard for real-time visualization.

1.3.3 3. Actuation Layer

- **DC Motors & Motor Driver:** Receive PWM signals from Arduino and adjust robot movement based on PID output for smooth line following.
- **Servo Motor & Camera:** Servo rotates the camera toward the detected card side (left or right) based on Arduino signal. Camera captures the card for identification.

1.3.4 4. Visualization Layer

- **Web Dashboard:**
  - Displays live video feed from the camera.
  - Shows detected object cards with labels and confidence scores.
  - Allows real-time monitoring of robot operation and card collection progress.

1.3.5 Signal Flow Summary

- **Line Following:** IR sensor → Arduino → DC motors
- **Card Detection:** Ultrasonic sensor → Arduino → Servo → Camera
- **Object Identification:** Camera → Raspberry Pi → Web dashboard

This architecture ensures that the servo mechanism reacts quickly and accurately to card detection while the robot continues line-following and obstacle avoidance operations seamlessly.

1.4 Data Flow / Assembly Description

The block diagram represents the interaction between sensing, processing, and actuation components of the robot. The signal flow through the system occurs in the following stages.

1.4.1 Sensor Assembly

The robot uses an IR sensor array to detect the line on the surface. These sensors generate electrical signals depending on whether they detect a black line or white background. The signals are transmitted to the Arduino microcontroller for further processing.

A camera module connected to the Raspberry Pi captures images of the surrounding environment. These images are used for object detection and identification.

1.4.2 Motor Driver Assembly

The Arduino sends PWM control signals to the motor driver module, which drives the DC motors to move the robot forward or adjust its direction for accurate line following.

1.4.3 Camera and Servo Assembly

The camera module is mounted on a servo motor which allows it to rotate and track objects.The Raspberry Pi controls a servo motor that adjusts the orientation of the camera so that it can track the detected object. The camera is connected to the Raspberry Pi through the CSI interface.

1.4.4 Power Distribution

The system uses two separate buck converters for stable power management. A 5V/3A buck converter supplies power to the Raspberry Pi, ensuring reliable operation for image processing and camera functions. The Arduino and associated sensors are powered by another 5V buck converter, which provides regulated voltage to the control circuitry.

Separating the power supplies helps prevent voltage fluctuations caused by motor operation from affecting the Raspberry Pi and improves the overall stability of the system.

1.5 Schematic

Schematic designed in EasyEDA showing the electrical connections between components.

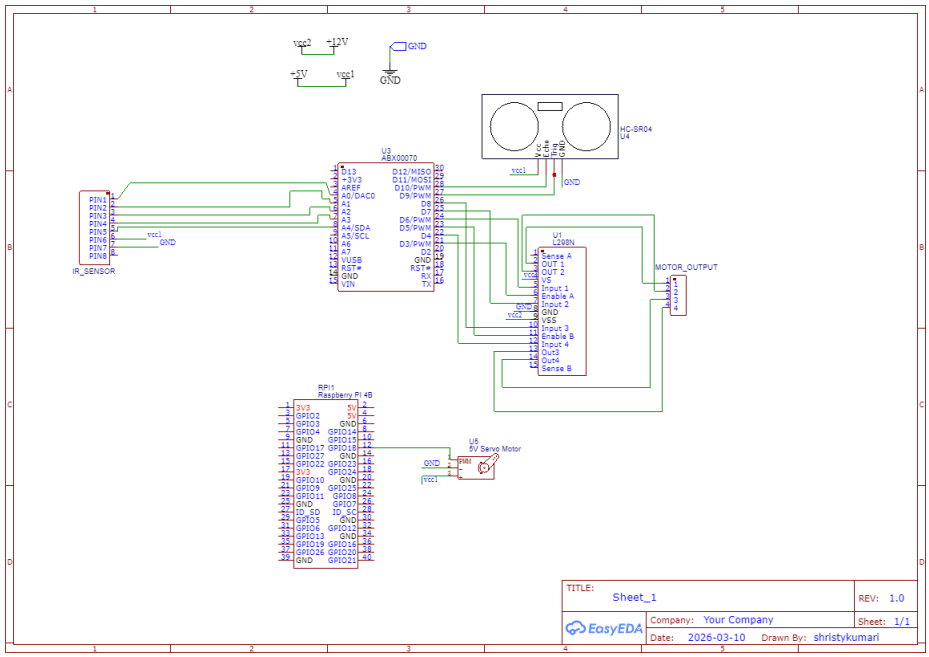

Figure 2: Circuit schematic of the system

1.6 Connection Description

The connections between the microcontrollers, sensors, and actuators are described below.

1. **IR Sensor Array to Arduino** The IR sensors are connected to the analog input pins of the Arduino for line detection.
   - S1 → A0
   - S2 → A1
   - S3 → A2
   - S4 → A3
   - S5 → A4
2. **Motor Driver to Arduino** The motor driver receives control signals from the Arduino digital pins.
   - ENA → Pin 3 (PWM)
   - IN1 → Pin 6
   - IN2 → Pin 7
   - ENB → Pin 5 (PWM)
   - IN3 → Pin 8
   - IN4 → Pin 4
3. **Ultrasonic Sensor Connection** The ultrasonic sensor is connected to the Arduino for obstacle detection.
   - TRIG → Pin 9
   - ECHO → Pin 10

- VCC → 5V
- GND → GND

4. **Servo Motor Connection** The servo motor used to rotate the camera is connected as follows:
   - Signal → PWM pin on Raspberry Pi / Arduino
   - VCC → 5V
   - GND → Ground
5. **Camera Module Connection** The camera module is connected directly to the Raspberry Pi through the CSI camera interface.
6. **Power Connections** Two buck converters are used for power regulation.
   - 5V/3A buck converter → powers the Raspberry Pi
   - Second buck converter → powers the Arduino and sensors

2 Software

2.1 Software Used

[noitemsep, topsep=0pt, left=0pt]

- Arduino IDE
- Python
- OpenCV
- EasyEDA
- LTspice
- GitHub
- Roboflow
- Raspberry OS
- Raspberry Imager

2.2 Line Following

The robot utilizes a combination of IR sensors, PID control, and non-blocking logic to follow a line and handle obstacles effectively. The key components and approaches are described below:

2.2.1 IR Sensor Array and Weighted Error

A 5-sensor IR array (S1–S5) is used to detect the black line on the surface. Each sensor outputs an analog value corresponding to the reflectivity of the surface beneath it. To compute the line position error, a **weighted sum** of the sensor readings is calculated as:

$$Error = \frac{\sum_{i=1}^{5} w_i \cdot S_i}{\sum_{i=1}^{5} S_i} - CenterPosition$$

where w_i represents the weight assigned to each sensor based on its position relative to the robot's center, and the center position corresponds to the ideal line location under the middle sensor. This error indicates the deviation of the robot from the line and serves as the input for the PID controller.

2.2.2 PID Control for Smooth Line Tracking

The robot uses a PID controller to minimize the line position error and adjust the motor speeds accordingly. The PID components include:

- **Proportional (P)**: Corrects the present error.
- **Derivative (D)**: Predicts future error to prevent overshoot.
- (Integral term not used in this implementation for faster response)

The PID output is applied to adjust the motor PWM differentially, enabling smooth steering. For our system, the chosen PID constants are:

$$K_p = 2, \quad K_d = 14$$

These values were selected through trial and error to minimize oscillations and maintain stable line following.

2.2.3 Line Recovery Function

The system uses a last-seen memory variable (lastSeen), which stores the position where the line was last detected. This value is derived from the weighted error calculated using the sensor array. Based on this stored information, the robot determines the most likely direction in which the line was lost.

If the line was last seen on the left, the robot rotates left; if it was last seen on the right, it rotates right. If the line was last detected near the center, the robot briefly moves backward to retrace its path. During this process, the robot continuously monitors the sensors.

2.2.4 Non-Blocking Obstacle Avoidance

An ultrasonic sensor monitors obstacles in the robot's path. The obstacle avoidance routine is non-blocking, which means:

- The robot can adjust speed or direction without stopping the PID control loop.
- Ultrasonic sensor continuously monitors the area in front of the robot while it performs normal PID-based line following. When an obstacle is detected within a predefined threshold distance, the robot stops its forward motion and performs a U-turn to change direction and move away from the obstacle. After turning around, the robot continues its operation and searches for the line again using the IR sensors, after which normal line-following behavior resumes.

2.2.5 Motor Driver Interface

Motors are controlled via an H-Bridge driver. The connections include:

- **ENA**/**ENB**: PWM enable pins for speed control.
- **IN1–IN4**: Direction control pins for forward and backward motion.

The PID output directly modulates the PWM signals to adjust motor speeds dynamically.

2.2.6 Overall Control Flow

The line following loop can be summarized as:

1. Read IR sensors and compute weighted error.
2. Calculate PID output and adjust motor PWM using K_p and K_d values.
3. Check for obstacles and modify trajectory if needed.
4. If the line is lost, invoke the line recovery function.
5. Repeat continuously in a non-blocking loop.

This approach ensures smooth line tracking, effective obstacle avoidance, and robust recovery in case of line loss.

2.3 Image Detection and Identification Pipeline

The onboard perception system processes images captured by the camera module to detect and identify objects in real time. The pipeline is divided into several stages:

2.3.1 Model Training

- The object detection model used in the system is trained using the YOLO (You Only Look Once) deep learning framework with a dataset hosted on Roboflow. The dataset is first downloaded directly to the Google Colab environment using the Roboflow API. This allows seamless access to the annotated images and automatically creates the required dataset folder structure in the Colab workspace.
- To train the detection model, the Ultralytics YOLO framework is installed. The training process begins by initializing the YOLO Nano model (yolo26n.pt), which is a lightweight and high-speed variant suitable for real-time applications and edge devices such as the Raspberry Pi.
- The model is trained for 100 epochs. Images are resized to 640×640 resolution, which maintains consistency with the preprocessing applied during dataset preparation. A batch size of 16 images per iteration is used to balance computational efficiency and GPU memory usage.
- After training, the resulting PyTorch model (.pt) is converted to the NCNN format, which is optimized for embedded systems. PyTorch requires a full Python runtime and significant computational overhead, which can be inefficient for devices like the Raspberry Pi. In contrast, NCNN is implemented in optimized C++, allowing faster inference and lower memory consumption on edge hardware.
- The trained model is then transferred to the Raspberry Pi and integrated into a lightweight vision script. This script captures frames from the camera, processes them through the trained YOLO model, and outputs detected object classes and confidence scores in real time.

2.3.2 Image Acquisition

- The camera module captures frames at a fixed resolution and frame rate.
- Each frame is stored in memory for processing.
- Preprocessing ensures consistent brightness and contrast for reliable detection.

2.3.3 Preprocessing

- Noise reduction using Gaussian or median filters.
- Conversion to grayscale or color space transformation (e.g., RGB to HSV) to simplify further processing.
- Optional thresholding or edge enhancement to highlight features of interest.

2.3.4 Region of Interest (ROI) Selection

- Defines areas within the frame where objects are expected.
- Reduces computational load by ignoring irrelevant parts of the image.
- Can be dynamically adjusted based on previous detections.

Figure 3: Arena

2.3.5 Object Detection

- Algorithms scan the ROI to locate potential objects. - Detection may use:

- Contour detection for shape-based objects.

- Color segmentation for colored markers or lines.
- Pre-trained lightweight object detection models for known object classes.

2.3.6 Feature Extraction

- Extracts key characteristics from detected objects such as:

- Shape, area, and aspect ratio.
- Color histograms or dominant color.
- Position coordinates relative to the robot.

Figure 4: web interface for live feed

2.3.7 Object Identification and Classification

- Identifies the type or class of the object using extracted features.
- Lightweight classifiers or rule-based logic are applied onboard to minimize processing delay.
- Outputs include object label, confidence score, and position.

2.3.8 Post-Processing and Action Trigger

- Filters false positives using size, location, or motion consistency checks. - Sends detection results to the control module for navigation or task execution. - Example actions:

- Adjust path to avoid obstacles.
- Follow a specific target object.
- Trigger event-based responses (e.g., pick-up or drop-off).

2.4 Web Dashboard

Making of Internal dashboard to monitor your bot's progress, track unique card detections, and view a live camera feed.

2.4.1 File structure

- bot dashboard: Main Directory, root folder that houses all your project files.
- templates: Sub-directory. Flask is hard-coded to look into a directory with this specific name to find your HTML files.
- index.html: Located inside the templates folder, this file acts as the "Communication Interface". It contains the layout for the live video feed, telemetry data like card counts, and the mission timer.

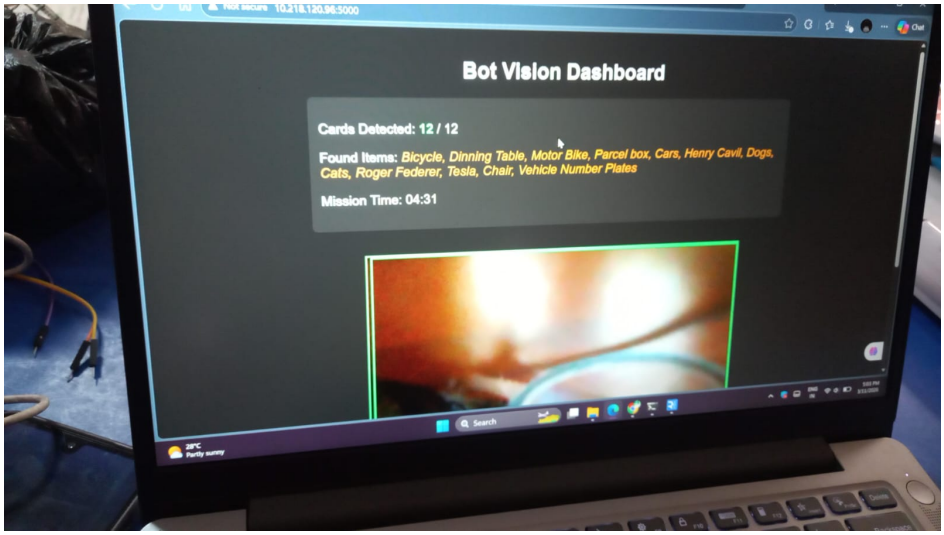

Figure 5: web interface for live feed

- app.py: Located in the root bot dashboard folder, this is the "Primary Brain" of the operation. It handles the heavy lifting: starting the Flask web server, running the YOLO object detection model, processing camera frames into a stream, and managing the mission logic.

Figure 6: web interface for live feed

2.5 Usage Documentation

This section describes the procedure required to operate the semi–autonomous robot system, including code deployment, system initialization, and execution of the vision-based card detection interface.

2.5.1 1. Uploading the Arduino Control Program

The Arduino microcontroller is responsible for low-level robot control, including line-following, motor actuation, ultrasonic sensing, and servo positioning. The Arduino program must first be uploaded before operating the robot.

1. Connect the Arduino board to a laptop using a USB cable.
2. Open the **Arduino IDE**.
3. Load the final control code for line-following, servo control, and sensor processing.
4. Select the correct board type and communication port.
5. Upload the code to the Arduino.
6. After successful upload, disconnect the laptop from the Arduino.

Once uploaded, the Arduino operates independently and can be powered directly through the Li-Po battery during robot operation.

2.5.2 2. Powering and Booting the Raspberry Pi

The Raspberry Pi is used for image processing and hosting the web-based monitoring interface.

1. Flash the Raspberry Pi OS onto a microSD card using the **Raspberry Pi Imager**.
2. Insert the microSD card into the Raspberry Pi.
3. Power the Raspberry Pi using a **5V** / **3A power bank**.
4. Allow the system to boot into the Raspberry Pi graphical interface.

The Raspberry Pi contains the object detection model, camera interface code, and the web dashboard used for monitoring card detection.

2.5.3 3. Launching the Vision and Web Interface

Once the Raspberry Pi has booted, the object detection server must be started.

1. Open a terminal in the Raspberry Pi GUI.
2. Navigate to the project directory containing the vision processing code.
3. Run the following command:

```
python3 app.py
```

This command launches a Flask-based web server which performs the following tasks:

- Captures live video frames from the Raspberry Pi camera.
- Runs the YOLO object detection model on each frame.
- Streams the processed video feed to a web browser.
- Maintains a record of detected image cards and elapsed runtime.

The server runs on port 5000 and can be accessed from any device connected to the same network.

2.5.4 4. Accessing the Web Dashboard

After starting the server, the monitoring interface can be opened from a laptop or mobile device.

1. Open a web browser.
2. Enter the Raspberry Pi's IP address followed by port 5000.

```
http://<raspberry_pi_ip>:5000
```

The web interface displays:

- Live camera video feed.

- Bounding boxes around detected image cards.
- Confidence scores of detected objects.
- Number of unique cards detected.
- Total time elapsed during the run.

A reset option is also provided in the interface to clear the memory of detected cards and restart the timer before beginning a new run.

2.5.5 5. Starting Robot Operation

After the web dashboard is active, the robot can be deployed.

1. Place the robot at the starting point of the track.
2. Use the **Reset** option on the web dashboard to clear previous detection data.
3. Power the Arduino and motor system using the **11.1V Li-Po battery**.

Once powered, the robot automatically begins its line-following operation.

2.5.6 6. Card Detection Process

During navigation, the robot continuously scans for image cards placed along the track.

- When an ultrasonic sensor detects the presence of a card on either side, the Arduino rotates the servo mechanism.
- The servo aligns the camera toward the detected card.
- The Raspberry Pi captures the image and processes it using the YOLO detection model.
- If a new card is detected with sufficient confidence, the system records the card and briefly pauses the scanning mechanism to stabilize detection.
- Detection results are immediately displayed on the web dashboard.

This process continues until the robot completes the track or all cards have been detected.

2.5.7 7. Resetting and Restarting

For repeated trials:

1. Press the **Reset** button on the web interface to clear previously detected cards.
2. Reposition the robot at the starting point.
3. Restart the robot by powering the motor system again.

This ensures that each run begins with a fresh detection log and timer.

1. Power the robot using the regulated power supply.
2. Upload the Arduino control code using Arduino IDE.
3. Run the image processing script on Raspberry Pi.
4. Place the robot on the track to start operation.

2.6 Bill of Materials (BOM)

S.No	Component	Specification	Qty	Description
1	Raspberry Pi 4 Model B	4GB RAM	1	Main processing unit responsible for running the vision model, camera control, and system logic.
2	Raspberry Pi Camera Module	CSI Interface	1	Captures images and video frames for the object detection system.
3	MG995 Servo Motor	360° Continuous Rotation	1	Rotates the camera platform to perform 360° environmental scanning.
4	CTR5000 IR Sensor Array	5 Channel	1	Detects black line patterns for line-following navigation.
5	Motor Driver	L298N	1	Controls the direction and speed of the DC motors using PWM signals.
6	DC Geared Motors	12V	2	Drives the wheels of the robot for movement and navigation.
7	Bot body	Differential Drive Platform	1	Structural frame that holds all mechanical and electronic components.
8	UBEC / Buck Converter	5V, 3A Output	1	Regulates battery voltage to a stable 5V supply for the Raspberry Pi.
9	Battery Pack	11.1V Li-Po	1	Primary power source for the entire robotic system.
10	Jumper Wires	Male-Male / Male-Female	Multiple	Used for electrical connections between components.
11	Breadboard	Prototyping Board	1	Used for circuit prototyping and testing connections.

Table 1: Bill of Materials used in the robotic system